Predefined Functional Interfaces

Java 8 Core Java java by devs5003 - May 16, 2021 0

I would consider that we have a clear understating of Functional Interfaces from the Topic "The Functional Interfaces in Java 8". We have also covered other topics related to it like Why is it introduced ?, its benefits & when to use it. However, some commonly used Predefined Functional Interfaces are listed in the attached table. Furthermore, we will discuss all of them in detail in below sections.

Function Type	Method Signature	Input parameters	Returns	When to use?
Predicate<T>	boolean test(T t)	one	boolean	Use in conditional statements
Function<T, R>	R apply(T t)	one	Any type	Use when to perform some operation & get some result
Consumer<T>	void accept(T t)	one	Nothing	Use when nothing is to be returned
Supplier<R>	R get()	None	Any type	Use when something is to be returned without passing any input
BiPredicate<T, U>	boolean test(T t, U u)	two	boolean	Use when Predicate needs two input parameters
BiFunction<T, U, R>	R apply(T t, U u)	two	Any type	Use when Function needs two input parameters
BiConsumer<T, U>	void accept(T t, U u)	two	Nothing	Use when Consumer needs two input parameters
UnaryOperator<T>	public T apply(T t)	one	Any Type	Use this when input type & return type are same instead of Function<T, R>
BinaryOperator<T>	public T apply(T t, T t)	two	Any Type	Use this when both input types & return type are same instead of BiFunction<T, U, R>

Table of Contents

What are Predefined Functional Interfaces ? Benefits Of Using them?

Java 8 has provided some Predefined (Built-in) Functional Interfaces to make our programming easier. Moreover, Predefined Functional Interfaces include most commonly used methods which are available to a programmer by default. In our day to day programming m

times we come across re-occurring functionalities to be developed. In that case, we can utilize the predefined functional interfaces instead of creating our own every time. They will obviously save our development time and minimize chances of mistakes.

Why are Predefined Functional Interfaces mandatory to learn?

Apart from Lambda expressions, we will be using predefined functional interfaces in Stream API most of the time. If you are going to learn Stream, we would suggest you to go through once with predefined functional interfaces. Additionally, if you are using Java 8 features in your project, then obviously you are going to use them. Furthermore, the Java community has started using predefined functional interfaces in new APIs. At least, it becomes very crucial to have the idea of method parameters & return type of each method in our mind. If you are not familiar with them, it will be very difficult to understand even new APIs in the language including Java Stream API.

Types of Predefined Functional Interfaces

Predicate<T>

We can use Predicate<T> to implement some conditional checks. However, from it's method signature : **boolean test(T t)** it is clear that it takes an input parameter and returns a Boolean result. When you have this type of requirement to write a method, use it confidently. Let's observe the method signature as below:

```
public interface Predicate<T> {
   boolean test(T t);
}

=> T denotes the input parameter type.
```

Example

For example, suppose we have to write a program to check if a number is a single digit number or not using Lambda expression. Since it's a conditional check, we will implement Predicate as below:

```
Predicate<Integer> p = (i) -> (i > -10) && (i < 10);
System.out.println(p.test(9));
```

Function<T, R>

Function<T, R> is used to perform some operation & returns some result. Unlike Predicate<T> which returns only boolean, Function<T, R> can return any type of value. Therefore, we can also say that Predicate is a special type of Function which returns only Boolean values.

```
interface Function<T,R> {
   R apply(T t);
}

=> T is method input parameter & R is return type
```

Example

For example, suppose we have to write a program to find length of a given String using Lambda expression. Since it's taking input, performing some operation & returning result, we will implement Function as below:

```
Function<String, Integer> f = s -> s.length();
System.out.println(f.apply("I am happy now"));
```

Consumer<T>

Consumer<T> is used when we have to provide some input parameter, perform certain operation, but don't need to return anything. Moreover, we can use Consumer to consume object and perform certain operation.

```
interface Consumer<T> {
  void accept(T t);
}

=> T denotes the input parameter type.
```

Example

For example, suppose we have to write a program to find length of a given String using Lambda expression. Since it's taking input, performing some operation & returning result, we will implement Consumer as below:

```
Consumer<String> c = s -> System.out.println(s);
c.accept("I consume data but don't return anything");
```

Supplier<R>

Supplier<R> doesn't take any input and it always returns some object. However, we use it when we need to get some value based on some operation like supply Random numbers, supply Random OTPs, supply Random Passwords etc. For example, below code denotes it :

```
interface Supplier<R>{
   R get();
}

=> R is a return type
```

Example

For example, suppose we have to write a program to supply 4 digit random OTPs using Lambda expression. Since it will return values without taking any input parameter, we will implement Supplier as below:

```
Supplier<String> otps = () -> {
    String otp = "";
    for (int i = 1; i <= 4; i++) {
      otp = otp + (int) (Math.random() * 10);
    }
  return otp;
};
System.out.println(otps.get());
System.out.println(otps.get());
System.out.println(otps.get());
```

BiPredicate<T, U>

BiPredicate<T, U> is same as Predicate<T> except that it has two input parameters. For example, below code denotes it:

```
interface BiPredicate<T, U> {
   boolean test(T t, U u)
}

=> T & U are input parameter types
```

Example

For example, suppose we have to check the sum of two integers is even or not by using BiPredicate, we will implement BiPredicate <T, U> as below:

```
BiPredicate<Integer,Integer> bp = (i,j)->(i+j) %2==0;
System.out.println(bp.test(24,34));
```

BiFunction<T, U, R>

BiFunction<T, U, R> is same as Function<T, R> except that it has two input parameters. For example, below code denotes it:

```
interface BiFunction<T, U, R> {
   R apply(T t, U u);
}

=> T & U are method input parameters & R is return type
```

Example

For example, suppose we have to find sum of two integers by using BiFunction, we will implement BiFunction <T,U,R> as below:

```
BiFunction<Integer,Integer,Integer> bf = (i,j)->i+j;
System.out.println(bf.apply(24,4));
```

BiConsumer<T, U>

BiConsumer<T> is same as Consumer<T> except that it has two input parameters. For example, below code denotes it:

```
interface BiConsumer<T, U> {
  void accept(T t, U u);
}

=> T & U are method input parameters
```

Example

For example, suppose we have to find concatenation of two strings & print result on the console by using BiConsumer, we will implement BiConsumer<T, U> as below:

```
BiConsumer<String,String> bc = (s1, s2)->System.out.println(s1+s2);
bc.accept("Bi","Consumer");
```

Other default & static methods of Predefined Functional Interfaces

However, It is just to remind from the concept of Functional Interfaces that in addition to a single abstract method they can also have static & default methods as well without violating the rules of Functional Interfaces. Consequently, here we will list all of them for each Functional Interface.

Predicate<T>

```
default Predicate<T> and(Predicate<? super T> other)
```

♦Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another.

```
static <T> Predicate<T> isEqual(Object targetRef)
```

⇒Returns a predicate that tests if two arguments are equal according to Objects.equals(Object, Object).

```
default Predicate<T> negate()
```

♦Returns a predicate that represents the logical negation of this predicate.

```
default Predicate<T> or(Predicate<? super T> other)
```

⇒Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another.

Function<T,R>

```
default <V> Function<T, V> andThen(Function<? super R, ? extends V> after)
```

⇒Returns a composed function that first applies this function to its input, and then applies the after function to the result.

```
default <V> Function<V, R> compose(Function<? super V, ? extends T> before)
```

♦Returns a composed function that first applies the before function to its input, and then applies this function to the result.

```
static <T> Function<T, T> identlity()
```

⇒Returns a function that always returns its input argument.

Consumer<T>

```
default Consumer<T> andThen(Consumer<? super T> after)
```

⇒Returns a composed Consumer that performs, then in sequence, this operation followed by the after operation.

TOP

Supplier<R>

There are no default & static methods in this interface.

BiPredicate<T, U>

```
default BiPredicate<T, U> and(BiPredicate<? super T, ? super U> other)
```

⇒Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another.

```
default BiPredicate<T, U> negate()
```

♦Returns a predicate that represents the logical negation of this predicate.

```
default BiPredicate<T, U> or(BiPredicate<? super T, ? super U> other)
```

⇒Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another.

BiFunction<T, U, R>

```
default <V> BiFunction<T, U, V> andThen(Function<? super R, ? extends V> after)
```

♦Returns a composed function that first applies this function to its input, and then applies the after function to the result.

BiConsumer<T, U>

```
default BiConsumer<T, U> andThen(BiConsumer<? super T, ? super U> after)
```

⇒Returns a composed BiConsumer that performs, in sequence, this operation followed by the after operation.

♥ Moreover, If you still want to learn more on Predefined Functional Interface, Kindly visit Oracle Documentation.

Tagged bifunction java 8 | built in functional interfaces | built-in functional interfaces in java | Built-in functional interfaces in java8 | Consumer | consumer in java 8 | Function | function in java 8 | Functional Interface Explained in Detail | functional interface java | functional interfaces in java | functional interfaces in java 8 | Java 8 Functional Interfaces | java function | Java Functional Interface | java functional interfaces | java predefined functional interfaces | predefined function | predefined functional interface in java | predefined functional interface in java 8 | Predefined Functional Interfaces | predefined functional interfaces in java | predefined functional interfaces in java 8 | predefined interface in java | predefined interfaces in java | Predicate | predicate in java 8 | Supplier | supplier java | what is predicate in java 8 | which of the following are built in functional interfaces | which one is predefined interface in java

‹ Previous article
Java Interview Questions and Answers

Next article ›
OOPs Design Principles

Name *

Email Address *

Website

Notify me of follow-up comments by email.

Notify me of new posts by email.

Post Comment

FOLLOW US

RECENT POSTS

- Spring Core MCQ and Answers Explained
- Spring MCQ and Answers Explained
- Why is Spring framework so popular ?
- Top 20 Collection Framework In Java Interview Questions
- Top 20 Java Interview Questions for OTT Platform Developers
- Record In Java With Examples
- Spring WebFlux CRUD Example
- 15 Frequently Asked Data Scientist Interview Questions
- Java 17 Features
- How To Resolve Common Java Exceptions

TOP